



Big O Notation Exercises

bigONotation.js

```
1 // 1. Even or odd
2
3 function isEven(value){
4     if (value % 2 == 0){
5         return true;
6     }
7     else
8         return false;
9 }
10
11 /*
12 Answer: O(1). Constant run time complexity.
13 Reasoning: Because you're only ever taking one value, there is no "loop" to go through.
14 Even as the value gets bigger, you simply divide it by 2 and see whether it returns an integer
15 */
16
17
18 // 2. Are You Here?
19
20 function areYouHere(arr1, arr2) {
21     //let ticks1, ticks2 = 0;
22     for (let i=0; i<arr1.length; i++) {
23         const el1 = arr1[i];
24         //ticks1++;
25         for (let j=0; j<arr2.length; j++) {
26             const el2 = arr2[j];
27             //ticks2++;
28             if (el1 === el2) return true;
29         }
30         //console.log(ticks1);
31         //console.log(ticks2);
32     }
33     return false;
34 }
35
36 /*
37 Answer: O(n^2). Quadratic run time complexity.
38 Reasoning: For each run through the first loop, you have to run through the entire second loop.
39 If you add even just one more item to `arr1`, you have to run another full loop through `arr2`.
40 So it's quadratic because as N doubles (taking N as `arr1.length` or `arr2.length`), the time i
```

```
41 will increase exponentially ( $N * N$ ).
42
43 UPDATE: As per the comments below, the above answer assumes somewhat naively that the two array
44 It's quite likely that they don't have the same lengths; therefore the complexity would be in f
45 */
46
47
48 // 3. Doubler
49
50 function doubleArrayValues(array) {
51     for (let i=0; i<array.length; i++) {
52         array[i] *= 2;
53     }
54     return array;
55 }
56
57 /*
58 Answer:  $O(n)$ . Linear run time complexity.
59 Reasoning: As `array.length` increases, the number of iterations increases at the same rate.
60 This is because you don't have to loop any more than once: however many items are in the array
61 is how many times you run the function.
62 */
63
64
65 // 4. Naive Search
66
67 function naiveSearch(array, item) {
68     for (let i=0; i<array.length; i++) {
69         if (array[i] === item) {
70             return i;
71         }
72     }
73 }
74
75 /*
76 Answer:  $O(n)$ . Linear run time complexity.
77 Reasoning: Same as above with the doubler. You have to check each and every item once and only
78 in order to determine whether you've got a match.
79 */
80
81
82 // 5. Creating Pairs
83
84 function createPairs(arr) {
85     //let ticks = 0;
86     for (let i = 0; i < arr.length; i++) {
87         for(let j = i+1; j < arr.length; j++) {
88             console.log(arr[i] + ", " + arr[j] );
89             //ticks++;
90         }
91     }
```

```

92     //console.log(ticks);
93 }
94
95 /* Answer:  $O(n^2)$ . Quadratic run time complexity.
96 Reasoning: The first loop has  $O(n)$  complexity, as with each new addition, the number of times
97 you run through the loop increases by one. As the inner loop also has  $O(n)$  complexity, together
98 quadratic run time complexity.
99 */
100
101
102 // 6. Computing Fibonacci Numbers
103
104 function generateFib(num) {
105     let result = [];
106     //let ticks = 0;
107     for (let i = 1; i <= num; i++) {
108         //ticks++;
109         if (i === 1) {
110             result.push(0);
111         }
112         else if (i == 2) {
113             result.push(1);
114         }
115         else {
116             result.push(result[i - 2] + result[i - 3]);
117         }
118     }
119     //console.log(ticks);
120     return result;
121 }
122
123 /* Answer:  $O(n)$ . Linear run time complexity.
124 Reasoning: As you add 1 to `num`, the run time complexity increases at the same rate.
125 */
126
127
128 // 7. Efficient Search
129
130 function efficientSearch(array, item) {
131     let minIndex = 0;
132     let maxIndex = array.length - 1;
133     let currentIndex;
134     let currentElement;
135
136     while (minIndex <= maxIndex) {
137         currentIndex = Math.floor((minIndex + maxIndex) / 2);
138         currentElement = array[currentIndex];
139
140         if (currentElement < item) {
141             minIndex = currentIndex + 1;
142         }

```

```

143         else if (currentElement > item) {
144             maxIndex = currentIndex - 1;
145         }
146         else {
147             return currentIndex;
148         }
149     }
150     return -1;
151 }
152
153 /* Answer:  $O(\log n)$ . Logarithmic run time complexity.
154 Reasoning: Cutting `array.length` in half in each iteration, the time complexity increases slowly.
155 */
156
157
158 // 8. Random element
159
160 function findRandomElement(arr) {
161     return arr[Math.floor(Math.random() * arr.length)];
162 }
163
164 /* Answer:  $O(1)$ . Constant run time complexity.
165 Reasoning: With no iteration occurring, selecting an element at random from an array has constant time complexity.
166 */
167
168
169 // 9. Is It Prime?
170
171 function isPrime(n) {
172     if (n < 2 || n % 1 !== 0) {
173         return false;
174     }
175     for (let i = 2; i < n; ++i) {
176         if (n % i === 0) return false;
177     }
178     return true;
179 }
180
181 /* Answer:  $O(n)$ . Linear run time complexity.
182 Reasoning: Disregarding the constant time it takes to check the first if condition, this function has a time complexity of  $O(n)$  as it iterates through each item once and only once.
183 */
184
185
186
187 // 10. Factorial of a number w/ recursion
188
189 function factorialOf(n) {
190     switch (n) {
191         case 0:
192         case 1:
193             return 1;

```

```
194         default: return n * factorialOf(n - 1);
195     }
196 }
197
198 /* Answer: O(n). Linear run time complexity.
199 Reasoning: This function is being called recursively n times before reaching the base case.
200 */
```



ben8p commented on Jan 7, 2019 • edited ▼

I think // 2. Are You Here? is $O(nm)$ where n is for array1 and m for array2
For each element of array1 we go to every element of array2
And length of array1 isn't related to array2



ArtOfBBQ commented on Apr 3, 2019

Thanks jhwheeler this was helpful to me~



stevearagonsite commented on Aug 4, 2019

Thanks, it's a great contribution , i needed exercises the Big(O) notation.



jhwheeler commented on Aug 5, 2019

Author

Glad to hear it helped you guys!



hgodinez89 commented on Jun 2, 2020

I agree with [@ben8p](#), the exercise number #2 is $O(arr1 * arr2)$ or $O(nm)$ because the function has two different inputs, and their lengths could very well be different.



azmym commented on Jun 27, 2020

I agree with [@ben8p](#) , length of arr1 not the same as arr2 so will be $O(n*m)$



jhwheeler commented on Jun 27, 2020

Author

I think // 2. Are You Here? is $O(nm)$ where n is for array1 and m for array2
For each element of array1 we go to every element of array2
And length of array1 isn't related to array2

Good point, if the two arrays have different lengths, then it would be $O(n*m)$ complexity instead of $O(n^2)$. Thanks for pointing this out, I'll update the gist to reflect this.



sommelon commented on Aug 23, 2020

Would be nice if there were some recursive functions



gitryder commented on Dec 16, 2020

[@jhwheeler](#), please add this recursive function to the gist

```
// 10. Factorial of a number w/ Recursion

function factorial0f(n) {
  switch (n) {
    case 0:
    case 1:
      return 1;
    default: return n * factorial0f(n - 1);
  }
}

/* Answer: O(n). Linear run time complexity.
Reasoning: This function is being called recursively n times before reaching the base
case.
*/
```



jhwheeler commented on Dec 17, 2020

Author

[@jhwheeler](#), please add this recursive function to the gist

```
// 10. Factorial of a number w/ Recursion

function factorial0f(n) {
  switch (n) {
    case 0:
    case 1:
      return 1;
    default: return n * factorial0f(n - 1);
  }
}
```

```
/* Answer: O(n). Linear run time complexity.
Reasoning: This function is being called recursively n times before reaching the
base case.
*/
```

@gitryder Done! Thank you for contributing :D



viniciusvasti commented on Mar 8, 2021

About // 2. Are You Here?

I agree that arrays inputs may be different.

But, isn't true that we analysis Big O thinking in the worst case?

The worst case would happens when both arrays have the same size.

Therefore I believe the Big O is $O(n^2)$.



adimitrova commented on Jun 22, 2021

I think // 2. Are You Here? is $O(nm)$ where n is for array1 and m for array2

For each element of array1 we go to every element of array2

And length of array1 isn't related to array2

The point is to get the worst-case scenario here. The worst-case scenario means $n==m$, therefore it will always be $O(N^2)$,



juandelacruz-calvo commented on Sep 21, 2021

This is very helpful, good compilation!



MichalNawrot commented on Oct 5, 2021

I believe in // 6. Computing Fibonacci Numbers time complexity should be $O(2^n)$ instead of $O(n)$.

[stackoverflow - computational-complexity-of-fibonacci-sequence](#)

[youtube - Big O Notation](#)

```
/* Answer: O(2^n). Exponential run time complexity. Reasoning: This function is calling
approximately two copies of itself recursively n times. */
```



juandelacruz-calvo commented on Oct 8, 2021

@MichalNawrot I think the solution posted is not recursive but iterative, it is using the previous Fibonacci number to calculate the next one, a much efficient solution than the ones you posted.

You can find an explanation of the differences here: https://www.youtube.com/watch?v=pqivnzmSbq4&ab_channel=mycodeschool



MichalNawrot commented on Oct 10, 2021 • edited ▼

@juandelacruz-calvo good catch, thanks for [a great link with the explanation](#). I should've read the code more carefully and noticed that `result.push(result[i - 2] + result[i - 3]);` doesn't recursively call `generateFib` function but rather add element to `result` list in linear manner.

It would be great to have here Fibonacci numbers calculated in both ways (recursive & iterative) to show the difference between $O(n)$ & $O(n^2)$, WDYT **@jhwheeler**?

Something along those lines:

```
// 6b. Computing Fibonacci Numbers (recursive)

function generateFibRecursive(num) {
  if(num < 2) {
    return num;
  }
  else {
    return generateFibRecursive(num - 1) + generateFibRecursive(num - 2);
  }
}

/* Answer:  $O(2^n)$ . Exponential run time complexity.
Reasoning: This function is calling approximately two copies of itself recursively n
times.
*/
```



juandelacruz-calvo commented on Oct 10, 2021

Having both solutions would be even better! Good idea! 🙌



CosmePantin commented on Oct 18, 2021

Thank you for the exercises! Does anyone have any good guide on how to identify algorithms that are $O(\log n)$ and $O(n \log n)$?

Are cubic algorithms $O(n^3)$ just three loops in one function?

I am trying to get exercises and examples of exponential algorithms $O(2^n)$ and factorial algorithms $O(n!)$



GonzaFarias commented on Apr 27, 2022

Gracias capo sos un re capo ayudaste a mi y a mis CUMPAS de la UNO - Saludos cumpa! Gonza!



CatrielNanthaveth commented on Apr 28, 2022

buenardo compi



CatrielNanthaveth commented on Apr 28, 2022

Gracias capo sos un re capo ayudaste a mi y a mis CUMPAS de la UNO - Saludos cumpa! Gonza!
q grande gonza



alizadafidan commented on Mar 13, 2023

Thank you, it was really helpful!



Muneeb552-star commented on Apr 29, 2023

[@jhwheeler](#) I think in Question #2, the time complexity will be $O(n^2)$ still.

Because, even if the two arrays have different lengths, the time complexity of the areYouHere function would still be $O(n^2)$ because the nested loops iterate through all possible pairs of elements in the two arrays, with n and m representing the lengths of the arrays, respectively. The fact that the arrays have different lengths doesn't change the fact that every element in `arr1` is compared with every element in `arr2`. Therefore, the time complexity would still be quadratic, which is $O(n^2)$.



Muneeb552-star commented on Apr 29, 2023

[@jhwheeler](#) Good point, if the two arrays have different lengths, then it would be $O(n*m)$ complexity instead of $O(n^2)$. Thanks for pointing this out, I'll update the gist to reflect this.

I think in Question #2, the time complexity will be $O(n^2)$ still.

Because, even if the two arrays have different lengths, the time complexity of the `areYouHere` function would still be $O(n^2)$ because the nested loops iterate through all possible pairs of elements in the two arrays, with n and m representing the lengths of the arrays, respectively. The fact that the arrays have different lengths doesn't change the fact that every element in `arr1` is compared with every element in `arr2`. Therefore, the time complexity would still be quadratic, which is $O(n^2)$.



abdoei commented on Dec 23, 2024

@jhwheeler Good point, if the two arrays have different lengths, then it would be $O(n*m)$ complexity instead of $O(n^2)$. Thanks for pointing this out, I'll update the gist to reflect this.

@Muneeb552-star

I think in Question #2, the time complexity will be $O(n^2)$ still.

Because, even if the two arrays have different lengths, the time complexity of the `areYouHere` function would still be $O(n^2)$ because the nested loops iterate through all possible pairs of elements in the two arrays, with n and m representing the lengths of the arrays, respectively. The fact that the arrays have different lengths doesn't change the fact that every element in `arr1` is compared with every element in `arr2`. Therefore, the time complexity would still be quadratic, which is $O(n^2)$.

It is not right as the worst-case scenario the function can get is iterating $N*M$ times. If you have $N = 3$, $M = 10$, you will iterate 30 times but $O(N^2)$ means that the maximum number of iterations you can, and you may, reach is 9 iterations.